

PROJECT REPORT

SMART COLLEGE EXPENSE TRACKER

A Full-Stack Student Financial Management System

Submitted By

Krishna Kushwaha

Registration Number: 25BAI11537

B.Tech – Computer Science and Engineering

VIT Bhopal University

Course: Programming in Java

1. Introduction

Smart College Expense Tracker is a full-stack fintech-style web application designed specifically for college students. It helps students record expenses, set budgets, analyze spending, manage recurring payments, split group bills, receive alerts, and generate reports.

The project combines a React/TypeScript frontend with a Java 17 Spring Boot backend. Spring Security and JWT protect authenticated operations, while Spring Data JPA/Hibernate provides persistence to MySQL. A browser simulator mode is also present for frontend preview.

The Student Spending Score is a distinctive rule-based 0–100 metric based on budget utilization, discretionary spending, category discipline, and retained savings.

2. Problem Statement

College students often manage spending across food, travel, education, subscriptions, hostel expenses, entertainment, and group purchases. Informal tracking makes it difficult to understand where money is going and how quickly a monthly allowance is being consumed.

The system centralizes these activities and provides budgeting, analytics, alerts, group settlement, recurring payments, and reporting in one application.

3. Objectives

Centralize student expense management; secure registration and login; manage expenses; maintain monthly and category budgets; visualize spending; calculate a transparent spending score; provide alerts; split group bills; manage recurring expenses; export financial reports; and demonstrate Java/Spring programming concepts.

4. Functional Requirements

Authentication: register, login, JWT session and authenticated profile.

Expenses: add, list, search/filter, update and delete.

Budget: overall monthly and category-specific budgets.

Analytics: category distribution, weekly spending and historical monthly trends.

Spending Score: transparent 0–100 financial-discipline metric.

Alerts: budget pace, spending spikes and buffer warnings.

Expense Splitter: participant shares, paid/owed values and net balances.

Recurring Expenses: weekly/monthly records with start/end dates.

Reports: monthly summaries, category audits and CSV export.

5. Non-Functional Requirements

Usability through responsive React UI; security through Spring Security, JWT and BCrypt; maintainability through layered packages; reliability through validation and controlled exceptions; data integrity through relational constraints; and portability through independent Vite and Maven workflows.

6. Technology Stack

Frontend: React 19, TypeScript, React Router, Vite, Tailwind CSS, Lucide React, Motion, Axios and Recharts.

Backend: Java 17, Spring Boot 3.2.3, Spring Security 6, JJWT 0.11.5, Spring Data JPA/Hibernate, MySQL 8.0, H2 and OpenCSV 5.9.

7. System Architecture

React Frontend → HTTP REST + JWT → Spring Security/JwtAuthenticationFilter → REST Controllers → Service Layer → Spring Data JPA Repositories → Hibernate → MySQL database.

8. Process Workflow

Start → Login/Register → JWT Authentication → Protected Dashboard → Select Module → REST Request → Controller → Service → Repository → Database → Response → Updated UI.

9. Design Diagrams

Use cases include managing expenses, setting budgets, viewing analytics, splitting bills, recurring expenses, reports, alerts and spending score.

Backend relationship: Controllers → Services → Repositories → Entities. Authentication: Login → AuthController → AuthService → BCrypt validation → JWT generation → protected requests.

10. Java and OOP Concepts Implemented

Classes and objects, encapsulation, exception handling, collections, constructors and methods, packages, REST programming, JPA/ORM and validation. The backend is explicitly structured into entity, repository, DTO, security, config, exception, service and controller packages.

11. Implementation Details

Frontend pages cover dashboard, expenses, add expense, budget, analytics, splitter, recurring expenses, reports, spending score, documentation, profile, login and registration. Reusable components include Navbar, Sidebar, ExpenseCard, BudgetProgress, SpendingScoreCard, AlertCard, modals, loading and toast components.

Backend classes include User, Expense, Budget, CategoryBudget, SharedExpense, SharedExpenseParticipant and RecurringExpense entities; repositories; DTOs; security classes; service classes; and REST controllers.

12. Database Design

The SQL schema defines users, expenses, budgets, category_budgets, shared_expenses, shared_expense_participants and recurring_expenses. Foreign keys maintain ownership relationships and unique constraints prevent duplicate monthly budget records.

13. API Design

Authentication: /api/auth/register, /login, /me. Expenses: /api/expenses. Dashboard: /api/dashboard. Budget: /api/budget and /api/budget/category. Analytics: /api/analytics, /monthly, /category. Score: /api/spending-score. Alerts: /api/alerts. Shared expenses: /api/shared-expenses. Recurring: /api/recurring-expenses. Reports: /api/reports and /api/reports/export.

14. Design Decisions and Rationale

React/TypeScript supports componentized UI; Spring Boot provides a structured Java REST backend; JWT supports stateless authentication; JPA/Hibernate simplifies persistence; MySQL provides relational storage; DTOs separate API payloads from entities; OpenCSV supports exports; and the score remains rule-based for transparency.

15. Results / Output

The source provides an integrated workflow for authentication, expenses, budgets, analytics, spending score, alerts, group bills, recurring payments, reports and profile/documentation. The dashboard and report modules provide

financial summaries, category audits and trend visualizations.

16. Testing Approach

Relevant verification includes registration/login, authenticated access, expense CRUD, budget updates, shared-bill calculations, recurring expenses, report generation, CSV export and rejection of unauthorized protected requests. The backend also includes Spring Boot/Security test dependencies and H2 for runtime testing.

17. Challenges Faced

Integrating many finance modules, maintaining authenticated state, modeling relational data, implementing split-bill calculations, designing an explainable score, coordinating REST layers/security, and keeping the UI modern but simple.

18. Learnings and Key Takeaways

The project provides practical experience with Java classes and OOP, Spring Boot layered architecture, JWT security, JPA/Hibernate, exception handling, DTOs, REST APIs, frontend state management, data visualization and financial calculations.

19. Future Enhancements

Possible enhancements include database migrations, role-based access, notifications, automated recurring transactions, optimized settlements, receipt OCR, statement import, forecasting, mobile support, cloud deployment and broader automated testing.

20. Conclusion

Smart College Expense Tracker applies Java and full-stack development concepts to a practical college-finance problem. It combines secure authentication, persistent storage, budgeting, analytics, score calculation, alerts, group settlement, recurring expenses and reports in a modular architecture that can be extended further.

21. References

Oracle Java Documentation; Spring Boot Documentation; Spring Security Documentation; Spring Data JPA/Hibernate Documentation; MySQL Documentation; React Documentation; TypeScript Documentation; Vite Documentation; Tailwind CSS Documentation; Recharts Documentation; OpenCSV Documentation; supplied project source and README.

22. Appendix

Project structure: frontend/src/components, context, pages, services and types; backend/src/main/java/com/college/expensetracker with entity, repository, dto, security, config, exception, service and controller packages; resources include application.properties, schema.sql and data.sql.

Student Spending Score components: Budget Utilization 35 points, Discretionary Spending Control 25, Category Discipline 20, Retained Savings Buffer 20. Total 100. Bands: 85–100 Excellent (A+), 75–84 Good (A), 60–74 Fair (B), 45–59 Needs Attention (C), below 45 Critical (D).

Run information from the supplied backend README: frontend development server on port 3000; backend on port 8080; MySQL database college_expense_db; Java 17+ and Maven 3.8+.